

I hate being wrong in public.

The Twelfth Doctor

In Chapter 3, we successfully reconstructed Digital Breast Tomosynthesis (DBT) images of synthetic volumes. However, we identified several limitations in our method, particularly concerning the reliability of the predictions generated by our post-processing neural network. Given the ill-posed nature of the problem, estimating the reliability of the solution would provide a degree of explainability. Moreover, the consistency of these reconstructions with the actual acquired X-ray projections is not assured, meaning that they might not be an actual solution to the inverse problem.

In this chapter, we aim to estimate the local uncertainty of the reconstructed volumes as well as ensuring data consistency. We first explore state-of-the-art methods for uncertainty quantification in deep learning in Section 4.1. Next, in Section 4.2, we enhance the quality of our synthetic training database to provide a better initialisation for our neural network. In Section 4.3, we modify our pipeline to improve the 2.5D training, generate uncertainty estimates, and enforce data consistency. Section 4.4 presents our results, focusing on image reconstruction and uncertainty estimates. We also verify the calibration of the uncertainty to the true error, and leverage this measure as a prior to ensure data consistency in the reconstructed volumes. Finally, Section 4.5 proposes an additional experiment leveraging these results. Specifically, we detail how uncertainty can be used to fuse information from networks trained in the three anatomical directions, thereby improving the overall results.

Some of the developments set out in this chapter were presented to the *International Symposium on Biomedical Imaging* (ISBI 2024) conference (Quillent et al., 2024a).

4.1 | Uncertainty in deep learning

In the literature, several uncertainty sources in the context of mathematical modelling are identified (Kiureghian et al., 2009; Gawlikowski et al., 2023). We propose here a partial list of these sources and give specific examples related to our DBT reconstruction task:

- Variability in the acquisition process: wrong positioning of the patient, environmental conditions;

- Errors in the measurement system: noise (electronic, quantum), defect of the detector, bad modelling of the X-ray acquisition process;
- Neural network's architecture: choice of the network's family (Generative Adversarial Network (GAN), auto-encoder, diffusion model...), too few or too many parameters;
- Bad optimisation: training stuck in a local minimum, use of an inappropriate loss function, suboptimal convergence.

The literature primarily distinguishes between two fundamental types of uncertainty: aleatoric and epistemic. These classifications provide a framework to analyse and quantify the various sources of uncertainty previously listed. In this section, we first introduce the theoretical foundations underlying Bayesian neural networks. Subsequently, we define both aleatoric and epistemic uncertainties. Finally, we present a comprehensive review of associated state-of-the-art methods.

4.1.1 | Bayesian Neural Networks (BNNs)

To perform our reconstruction task, we seek to train a neural network in a supervised fashion on a dataset $\mathcal{D} = \{(\mathbf{x}_t, \mathbf{x}_t^*)\}_{t=1}^T$ comprised of T pairs of initial reconstructed volumes $\mathbf{x}$ and ground truth phantoms $\mathbf{x}^*$. We consider that the reconstructions and the phantoms are actually vectors of voxels, *i.e.*, $\mathbf{x}_t, \mathbf{x}_t^* \in \mathbb{R}^n$. The network depends on $\theta \in \Theta$, a set of parameters optimised during the training process.

Throughout this chapter, we adopt a Bayesian perspective on neural networks. In the Bayesian framework, we seek to estimate probability distributions over the weights and data rather than getting point estimates (Jospin et al., 2022). From a Bayesian point of view, inference involves estimating the posterior predictive distribution

$$p(\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}) = \int_{\Theta} p(\hat{\mathbf{x}}'|\mathbf{x}', \theta) p(\theta|\mathcal{D}) d\theta \quad (4.1)$$

for a new input $\mathbf{x}'$ (for instance, a sample from the test set), given the prior knowledge acquired from $\mathcal{D}$. It is important to note that this marginalisation process provides us with a full distribution over the predicted values $\hat{\mathbf{x}}'$. The likelihood $p(\hat{\mathbf{x}}'|\mathbf{x}', \theta)$ captures the uncertainty about the data. It includes all possible output predictions given the learnt weights θ and the input $\mathbf{x}'$. The posterior $p(\theta|\mathcal{D})$ denotes all possible learnt weights given the training dataset, and models the uncertainty about the parameters that stems from the randomised nature of the optimisation process.

The reconstruction can be computed as the expected value $\mathbb{E}[\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}]$ of the posterior predictive distribution. The total uncertainty, for its part, is commonly measured by the variance

of the posterior predictive distribution, $\text{Var}[\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}]$. Resorting to the law of total variance, we can decompose the uncertainty as

$$\text{Var}[\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}] = \underbrace{\mathbb{E}_{p(\theta|\mathcal{D})}[\text{Var}(\hat{\mathbf{x}}'|\mathbf{x}', \theta)]}_{\text{Aleatoric uncertainty}} + \underbrace{\text{Var}_{p(\theta|\mathcal{D})}[\mathbb{E}(\hat{\mathbf{x}}'|\mathbf{x}', \theta)]}_{\text{Epistemic uncertainty}}. \quad (4.2)$$

The aleatoric term corresponds to the part of uncertainty inherent to the data, while the epistemic term estimates the uncertainty that is due to the model parameters (Kiureghian et al., 2009).

To obtain both the reconstruction and the associated predictive uncertainty, we need to perform inference using the predictive posterior $p(\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D})$. However, Equation (4.1) presents two significant challenges. First, the integration is carried out over all possible weights θ in the parameter space Θ , an operation that is analytically infeasible. Second, using the Bayes rule,

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})},$$

we observe that the posterior depends on the evidence $p(\mathcal{D})$ which is intractable but constant with respect to θ . Thus, the posterior predictive distribution from Equation (4.1) can only be approximated. In the literature, the most common solution is to use Monte Carlo methods (Metropolis et al., 1949):

$$p(\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}) \approx \frac{1}{K} \sum_{k=1}^K p(\hat{\mathbf{x}}'|\mathbf{x}', \theta_k) \quad \text{with} \quad \theta_k \sim p(\theta|\mathcal{D}), \quad (4.3)$$

which still requires to sample from the intractable posterior.

In the remainder of this section, we review several references regarding the modelling of the data likelihood $p(\hat{\mathbf{x}}'|\mathbf{x}', \theta)$ with a neural network, and thus related to the estimation of aleatoric uncertainty (Section 4.1.2). We then study different approximations of the weights posterior $p(\theta|\mathcal{D})$ required to retrieve epistemic uncertainty (Section 4.1.3).

4.1.2 | Aleatoric uncertainty

During inference, aleatoric uncertainty stems from the data likelihood $p(\hat{\mathbf{x}}'|\mathbf{x}', \theta)$ and represents the inherent variability of the observed phenomenon. This type of uncertainty cannot be reduced by adding more data or refining the model, as it is intrinsic to the problem itself. It corresponds to the portion of uncertainty that cannot be explained even by a perfect modelling system. In our DBT case, aleatoric uncertainty occurs because the solution to the inverse problem admits multiple solutions for a single set of projections. Therefore, it could only be mitigated by adopting a different acquisition protocol.

In classic supervised deep learning, the weights of the neural network can be optimised using Maximum Likelihood Estimator (MLE) during the training phase:

$$\begin{aligned}
\theta_{\text{MLE}} &= \arg \max_{\theta} p(\mathcal{D}|\theta) \\
&= \arg \max_{\theta} \prod_{t=1}^T p(\mathbf{x}_t^*|\mathbf{x}_t, \theta) \\
&= \arg \max_{\theta} \prod_{t=1}^T \prod_{i=1}^n p(x_{ti}^*|\mathbf{x}_t, \theta), \tag{4.4}
\end{aligned}$$

where x_{ti}^* denotes the i -th voxel of the t -th vectorised ground truth volume in the training dataset. Here, we assume that all voxels within each volume are independent. Remark that the probability of obtaining the specific ground truth voxel x_{ti}^* depends on the whole image $\mathbf{x}_t$ as the neural network uses information coming from the neighbouring voxels to make its predictions. Using the MLE, we seek to estimate the weights of the neural network such that the probability of obtaining the known ground truth $\mathbf{x}_t^*$ from the initial reconstruction $\mathbf{x}_t$ is maximal. The likelihood $p(x_{ti}^*|\mathbf{x}_t, \theta)$ is set to a well-known distribution, commonly a Gaussian or Laplace distribution. Furthermore, the scale parameter of these distributions is considered constant for all voxels in the dataset, an assumption known as homoscedasticity (Yang et al., 2019). Hence, in the case of a Gaussian distribution, the MLE reduces to a Mean Squared Error (MSE) loss function (L2 norm), and in the case of a Laplace distribution, it reduces to a Mean Absolute Error (MAE) loss function (L1 norm).

Usually, the scale parameter could also be estimated during optimisation under the heteroscedastic hypothesis, *i.e.*, considering the scale parameter as a variable. Kendall et al. (2017), building on the work of Nix et al. (1994), propose to model both the location and scale parameters as outputs of the neural network. The neural network can thus be seen as modelling $p(\mathbf{x}'|\mathbf{x}', \theta)$ by predicting its characteristic parameters: as the density is now known, we can sample from it. In the field of image restoration, most authors choose to model the data likelihood using a Gaussian distribution (Kendall et al., 2017; Schlemper et al., 2018b; Laves et al., 2020; Tölle et al., 2021), a few using a Laplace distribution (Nair et al., 2022; Baumann et al., 2023; Angermann et al., 2023), and some researchers even propose more exotic distributions like generalised Gaussian (Upadhyay et al., 2021).

For a Gaussian distribution, the heteroscedastic likelihood takes the form:

$$p(x_{ti}^*|\mathbf{x}_t, \theta) = \mathcal{N}(x_{ti}^*|f_{\mu}(\mathbf{x}_t)_i, f_{\sigma^2}(\mathbf{x}_t)_i).$$

$f_{\mu}(\mathbf{x}_t)$ and $f_{\sigma^2}(\mathbf{x}_t)$ can be seen as ‘maps’ where the voxel at index i corresponds to the mean and variance predicted by the network for x_{ti} , the i -th voxel of $\mathbf{x}_t$. For notational convenience, we

denote $f_\mu(x_{ti}) \triangleq f_\mu(\mathbf{x}_t)_i$ and $f_{\sigma^2}(x_{ti}) \triangleq f_{\sigma^2}(\mathbf{x}_t)_i$. The explicit form of the likelihood is then given by

$$p(x_{ti}^*|\mathbf{x}_t, \theta) = \mathcal{N}(x_{ti}^*|f_\mu(x_{ti}), f_{\sigma^2}(x_{ti})) = \frac{1}{\sqrt{2\pi f_{\sigma^2}(x_{ti})}} \exp\left(-\frac{(x_{ti}^* - f_\mu(x_{ti}))^2}{2f_{\sigma^2}(x_{ti})}\right).$$

Nevertheless, computing the MLE from Equation (4.4) can be numerically unstable as it involves multiplying numerous exponential terms. Moreover, deep learning optimisation algorithms typically operate as minimisers (*e.g.*, the stochastic gradient descent). It is therefore conventional to employ the Negative Log-Likelihood (NLL), which transforms the maximisation into a minimisation problem and converts the product into a sum:

$$\begin{aligned} \arg \max_{\theta} \prod_{t=1}^T \prod_{i=1}^n p(x_{ti}^*|\mathbf{x}_t, \theta) &= \arg \min_{\theta} \left(-\log \prod_{t=1}^T \prod_{i=1}^n p(x_{ti}^*|\mathbf{x}_t, \theta) \right) \\ &= \arg \min_{\theta} \left(-\sum_{t=1}^T \sum_{i=1}^n \log p(x_{ti}^*|\mathbf{x}_t, \theta) \right) \\ &= \arg \min_{\theta} \sum_{t=1}^T \sum_{i=1}^n \frac{(x_{ti}^* - f_\mu(x_{ti}))^2}{2f_{\sigma^2}(x_{ti})} + \frac{1}{2} \log f_{\sigma^2}(x_{ti}) + \frac{1}{2} \log 2\pi \\ &= \arg \min_{\theta} \underbrace{\sum_{t=1}^T \sum_{i=1}^n \frac{1}{2} \left(\frac{(x_{ti}^* - f_\mu(x_{ti}))^2}{f_{\sigma^2}(x_{ti})} + \log f_{\sigma^2}(x_{ti}) \right)}_{\mathcal{L}_{\text{L2-NLL}}}. \end{aligned} \quad (4.5)$$

Using $\mathcal{L}_{\text{L2-NLL}}$ from Equation (4.5) as a loss function enables the simultaneous training of both output branches f_μ and f_{σ^2} . As observed by Kendall et al. (2017), the NLL acts as an adaptive attenuation mechanism for the loss function. The $\log f_{\sigma^2}(x_{ti})$ term constrains the network from producing excessive uncertainty values that would otherwise minimise the squared error fraction. When $f_{\sigma^2}(x_{ti})$ is treated as a constant, Equation (4.5) simplifies to the standard MSE loss function. The likelihood for a heteroscedastic Laplace distribution is derived later in this chapter, in Section 4.3.1.

4.1.3 | Epistemic uncertainty

Epistemic uncertainty arises from the posterior distribution of the weights $p(\theta|\mathcal{D})$ and reflects the variability and approximations inherent in the modelling system (in this case, the neural network). This type of uncertainty can be mitigated by acquiring more data or improving the model.

As discussed in Section 4.1.1, the posterior $p(\theta|\mathcal{D})$ is intractable. To address this, several authors have proposed methods for approximate Bayesian inference using specialised algorithms.

In this section, we focus our review on the principal methods, specifically Variational Inference (VI) and Deep Ensembles (DEs).

Variational inference The idea behind VI is to approximate the posterior $p(\theta|\mathcal{D})$ by a well-known tractable distribution $q_\phi(\theta)$ parametrised by ϕ (Blei et al., 2017). These parameters are learnt as part of the optimisation process so that $q_\phi(\theta)$ is close to the true posterior. This is effectively done by minimising the Kullback-Leibler divergence (Kullback et al., 1951):

$$\text{KL}[q_\phi(\theta)||p(\theta|\mathcal{D})] = \int_{\Theta} q_\phi(\theta) \log \frac{q_\phi(\theta)}{p(\theta|\mathcal{D})} d\theta = \mathbb{E}_{q_\phi(\theta)} \left[\log \frac{q_\phi(\theta)}{p(\theta|\mathcal{D})} \right].$$

However, as the KL-divergence relies on the intractable $p(\theta|\mathcal{D})$, in practice, the Evidence Lower Bound (ELBO) is used instead (Kingma et al., 2013),

$$\text{ELBO}(\theta) = \mathbb{E}_{q_\phi(\theta)} \left[\log \frac{p(\theta, \mathcal{D})}{q_\phi(\theta)} \right].$$

The ELBO is equal to the KL-divergence up to a constant:

$$\text{KL}[q_\phi(\theta)||p(\theta|\mathcal{D})] = -\text{ELBO}(\theta) + \log p(\mathcal{D}),$$

since $p(\theta, \mathcal{D}) = p(\theta|\mathcal{D})p(\mathcal{D})$. Thus, minimising the divergence is equivalent to maximising the ELBO:

$$\arg \min_{\theta} \text{KL}[q_\phi(\theta)||p(\theta|\mathcal{D})] = \arg \min_{\theta} \underbrace{\mathbb{E}_{q_\phi(\theta)} [\log q_\phi(\theta) - \log p(\theta, \mathcal{D})]}_{-\text{ELBO}(\theta)}.$$

This ELBO is then optimised using stochastic variational inference (Hoffman et al., 2013), which essentially applies stochastic gradient descent to VI. Blundell et al. (2015) developed the Bayes-by-Backprop algorithm, which combines stochastic variational inference with a reparametrisation trick (inspired by variational auto-encoders) to facilitate the backpropagation of the ELBO. This technique allows VI to be effectively utilised in modern deep learning frameworks. However, Bayes-by-Backprop necessitates storing and optimising additional parameters, leading to increased computational demands.

Monte Carlo Dropout Gal et al. (2016), the original authors of Monte Carlo Dropout (MCDO), expanded on the dropout concept introduced by Srivastava et al. (2014). They derive the method from variational inference, although it is merely an approximation. MCDO treats $q_\phi(\theta)$ as multiplicative Bernoulli noise applied to the network's weights, which remains active during inference. This approach can be seamlessly integrated into any network architecture and often results in faster and more efficient training compared to Bayes-by-Backprop. Extending

MCDO, Kingma et al. (2015b) presented variational dropout, a learnt generalisation of Gaussian dropout. Finally, Kendall et al. (2017) combined a Gaussian NLL with MCDO to estimate both aleatoric and epistemic uncertainties.

Numerous studies have applied variational inference or its approximations to inverse problems in imaging, particularly for medical image reconstruction. Barbano et al. (2020) addressed Computed Tomography (CT) reconstruction using a single Bayesian layer in a Convolutional Neural Network (CNN). Tölle et al. (2021) experimented with Bayes-by-Backprop for inpainting, super-resolution, and denoising across various imaging modalities, while Sudarshan et al. (2022) investigated variational dropout for positron emission tomography. Additionally, due to the simplicity of implementing MCDO techniques, this approach to variational inference is widely represented in the literature. Schlemper et al. (2018b) used MCDO to reconstruct Magnetic Resonance Imaging (MRI) volumes, and Laves et al. (2020) applied it to denoise ultrasound and optical tomography images. Finally, Shang et al. (2023) explored Bernoulli, Laplace, and Gaussian likelihoods incorporated into an MCDO network for single-pixel imaging.

Deep ensembles Deep Ensembles were proposed by Lakshminarayanan et al. (2017). This approach is based on the idea that a group of deep neural networks yields better predictions than a single model. As explained by Wilson et al. (2020), single deterministic models and BNNs converge to a unique local optimum of the loss landscape. Although BNNs account for the uncertainty around this local optimum, both methods yield similar parameter samples at each forward pass. By contrast, each member of a DE converges to a different basin of attraction, providing more variability to the parameters. DEs are often not considered as Bayesian methods as they are purely deterministic. However, several authors demonstrated that DEs can actually be seen as approximate Bayesian inference, sometimes outperforming true Bayesian methods (Wilson et al., 2020; Gustafsson et al., 2020; Pearce et al., 2020). Indeed, as stated in Equation (4.3), the predictive posterior can be calculated by Monte-Carlo integration: $p(\hat{\mathbf{x}}'|\mathbf{x}', \mathcal{D}) \approx \frac{1}{K} \sum_{k=1}^K p(\hat{\mathbf{x}}'|\mathbf{x}', \theta_k)$. Taking θ_k as the weights of an ensemble member k results in proper Bayesian inference with DEs. From there, it is straightforward to estimate epistemic uncertainty using Equation (4.2).

The performance of DEs for uncertainty quantification depends on the variability of the weights between each member. Altarabichi et al. (2024) show that in most cases, random initialisation of the network's parameters is sufficient to ensure that each member will converge to a different optimum. Besides, MCDO networks can also be loosely seen as an alternative form of ensemble, as dropping neurons during inference yields slightly different architectures for each prediction.

However, constructing a DE is computationally demanding as it necessitates the independent training of multiple networks and several forward passes during inference, which may require parallelisation across multiple computing machines. Reducing the number of models in the ensemble alleviates this constraint at the expense of diminishing the expressive power and thus the accuracy of the uncertainty estimation. To address this issue, various methods have been proposed in the literature. Some approaches concentrate on altering the training procedure. Snapshot ensembles (Huang et al., 2017) involves saving the weights of the network at different stages of the training process and using each checkpoint as an ensemble member. Yet, the parameters generated with this technique lack variability as they are not always sampled from local optima. Stochastic weight averaging (SWA) (Izmailov et al., 2018) reuses the same idea but averages the weights around a local optimum to create a single DE member that is more accurate. SWAG (Maddox et al., 2019) improves on this concept by considering each weight sampled after convergence as coming from a Gaussian distribution. MultiSWAG (Wilson et al., 2020) applies SWAG-like averaging across multiple models with diverse initialisations.

Moreover, distillation strategies have emerged, proposing to alter the network’s architecture to consolidate the knowledge of an ensemble into a single model. In this category, BatchEnsemble (Wen et al., 2020) models the weights as a combination of specific per-member variables and shared parameters. In addition, PackedEnsemble (Laurent et al., 2022) creates multiple independent networks within the same model that are jointly trained. Eventually, Havasi et al. (2020) devised MIMO, a model configuration where a single network’s capacity is used to train several sub-networks independently learning the desired objective.

Despite the extensive theoretical works on DEs, their application in medical imaging remains relatively limited, particularly in the context of reconstruction. For instance, Yang et al. (2023) utilised DEs to predict molecular properties and their associated uncertainty. Küstner et al. (2024) assessed both aleatoric and epistemic terms for MRI reconstruction using various deep learning architectures. Lastly, Baumann et al. (2023) employed a modified MIMO framework, which includes multiple encoders and decoders sharing a common latent space, for pixel regression tasks outside the medical field.

In this section, we reviewed several prevalent methods for estimating uncertainty in deep learning. Researchers typically differentiate between uncertainty inherent to the problem itself (aleatoric) and uncertainty arising from the optimisation process of the neural network (epistemic). The aleatoric component is modelled using a specifically chosen probability distribution, which the neural network approximates. Its estimation involves adding a second output to the neural network and using a heteroscedastic NLL to train the model.

Estimating the epistemic component, on the other hand, requires more intricate implement-

ation. Variational inference methods provide theoretically sound estimates but are challenging to train and computationally intensive. Consequently, many researchers prefer approximations such as Monte Carlo dropout and deep ensembles. These two methods also offer simpler interpretability.

In Section 4.3, we will implement both MCDO and DE to estimate aleatoric and epistemic uncertainty. However, we will favour the use of a Laplace distribution over the less robust Gaussian setting that most prior works employ.

4.2 | Improvement of the synthetic training database

In Chapter 3, we built our reconstruction pipeline on top of an Filtered Back-Projection (FBP) volume. We now improve this starting point by replacing it with an iterative reconstruction that converges towards the pseudo-inverse of the projection operator, which better minimises data consistency. Yet, implementing such iterative methods with the DBT system’s digital twin used in Chapter 3 is challenging due its memory cost. We thus swap the digital twin with a more flexible Graphics Processing Unit (GPU)-based algorithm. Taking advantage of this new pipeline, we increase the size of the training database by a factor of 2.5.

4.2.1 | GPU projector

In Section 3.2.2, we utilised the digital twin of a DBT system to simulate the projections of our phantoms (De Man et al., 2007; Sánchez de la Rosa et al., 2019). This software, developed and rigorously validated by GE HealthCare (GEHC), accurately models the various noise and scatter components of the projection process. The simulated measurements are provided in photon counts, closely mimicking the behaviour of a real imaging system. However, this digital twin has several limitations. It is cumbersome to handle and cannot be easily integrated into an end-to-end database creation pipeline. The projection simulation, computed on CPU, is very time-consuming. Additionally, due to the high computational cost, it cannot process large volumes. The digital twin also requires a specific normalisation procedure to retrieve attenuation coefficients (see Section 3.2.2). Furthermore, it cannot perform back-projection and necessitates other software for this task.

Consequently, we sought to replace the digital twin with a more flexible GPU projector/back-projector to create a proof of concept. The digital twin nonetheless remains the optimal choice for thorough evaluations that are beyond the scope of this thesis. Several GPU projectors are freely available online (van Aarle et al., 2016; Biguri et al., 2016; Kim et al., 2023). For an experimental internal project, the teams at GEHC also developed a similar algorithm in collaboration